

# Ticket 01 Study Report: Black-Scholes Utilities

SURF2026 American Option Risk Surfaces

Codex-assisted implementation report

June 15, 2026

## Abstract

Ticket 1 builds the analytic foundation for later solver validation: payoff functions and European Black-Scholes closed-form pricing utilities with continuous dividends. This report is written for beginner researchers with accounting, data, machine-learning, and Python experience who are still new to American option pricing, PDE methods, CN/PSOR, free-boundary problems, and Greeks. It explains why this small analytic layer comes before finite differences, records the implementation and tests, and states clearly that Ticket 1 does not validate any future CN/PSOR solver.

## Contents

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Role of This Ticket in the Whole Project</b>       | <b>3</b> |
| <b>2</b> | <b>Finance and Mathematical Background</b>            | <b>3</b> |
| 2.1      | Call and Put Payoffs . . . . .                        | 3        |
| 2.2      | Black-Scholes Inputs with Dividends . . . . .         | 4        |
| 2.3      | Closed-Form European Prices . . . . .                 | 4        |
| 2.4      | Put-Call Parity . . . . .                             | 4        |
| 2.5      | Zero-Volatility Deterministic Case . . . . .          | 5        |
| 2.6      | Why This Is Not American Option Solving Yet . . . . . | 5        |
| <b>3</b> | <b>Design Decisions</b>                               | <b>5</b> |
| 3.1      | Package Namespace . . . . .                           | 5        |
| 3.2      | Testing Framework . . . . .                           | 5        |
| 3.3      | Avoiding SciPy for Now . . . . .                      | 5        |
| 3.4      | Scalar and Array Inputs . . . . .                     | 6        |
| 3.5      | Explicit Edge Cases . . . . .                         | 6        |
| <b>4</b> | <b>Implementation Record</b>                          | <b>6</b> |
| 4.1      | Files Created . . . . .                               | 6        |
| 4.2      | Public Functions . . . . .                            | 6        |

|           |                                        |           |
|-----------|----------------------------------------|-----------|
| 4.3       | Function Notes . . . . .               | 7         |
| <b>5</b>  | <b>Testing Strategy</b>                | <b>7</b>  |
| <b>6</b>  | <b>Test Results and Interpretation</b> | <b>8</b>  |
| <b>7</b>  | <b>Limitations</b>                     | <b>9</b>  |
| <b>8</b>  | <b>Lessons Learned / Study Notes</b>   | <b>9</b>  |
| <b>9</b>  | <b>Link to Next Ticket</b>             | <b>10</b> |
| <b>10</b> | <b>Reproducibility Record</b>          | <b>10</b> |
| 10.1      | Files Changed . . . . .                | 10        |
| 10.2      | Commands . . . . .                     | 10        |
| 10.3      | Git Status Summary . . . . .           | 11        |

# 1 Role of This Ticket in the Whole Project

Ticket 1 is the first coding ticket after the setup check. Its job is intentionally small: build reliable payoff functions and European Black-Scholes closed-form pricing utilities before the project starts finite-difference or CN/PSOR work.

This order matters. Later tickets will build a numerical solver that approximates option values on a grid. A numerical solver is useful only if we can compare it against trusted reference values. For European options under the Black-Scholes model with continuous dividends, closed-form formulas already exist (Black and Scholes, 1973; Merton, 1973). That makes European calls and puts an ideal benchmark for the future solver's first validation step.

In plain language, Ticket 1 gives the project two kinds of building blocks:

- payoff functions, which describe what an option is worth if exercised at a given spot price;
- European closed-form prices, which give analytic benchmark values before any American early exercise logic is introduced.

The solver roadmap says Ticket 1 must come before finite differences because later failures should be easy to diagnose. If a future Crank-Nicolson result disagrees with a European closed-form price, we need confidence that the closed-form comparison is correct. Otherwise we would not know whether the bug is in the numerical grid, the PDE operator, the boundary conditions, or the analytic benchmark itself.

This ticket does not validate the future CN/PSOR solver. It prepares one trusted analytic layer that the solver validation plan will use later, especially in the European closed-form check. The correct project habit is disciplined layering: validate the simple analytic layer before trusting more complicated numerical layers.

## 2 Finance and Mathematical Background

An option is a contract whose value depends on an underlying asset price, called  $S$ . The strike price, called  $K$ , is the exercise price written into the contract.

### 2.1 Call and Put Payoffs

A call option gives the holder the right to buy the asset for  $K$ . If the market spot price is above  $K$ , the right to buy cheaply is valuable. If the spot is below  $K$ , exercising the call would not help. The call payoff is

$$\Phi_{\text{call}}(S) = \max(S - K, 0). \quad (1)$$

A put option gives the holder the right to sell the asset for  $K$ . If the market spot price is below  $K$ , the right to sell at the higher strike is valuable. If the spot is above  $K$ , exercising the put would not help. The put payoff is

$$\Phi_{\text{put}}(S) = \max(K - S, 0). \quad (2)$$

The payoff is also called intrinsic value. It is what the option is worth if exercised immediately. For a European option, exercise is allowed only at maturity. Therefore, when  $T = 0$ , a European option price must equal its payoff.

## 2.2 Black-Scholes Inputs with Dividends

The Black-Scholes model with continuous dividends uses the following notation:

| Symbol   | Meaning                                           |
|----------|---------------------------------------------------|
| $S$      | Current spot price of the underlying asset.       |
| $K$      | Strike price.                                     |
| $T$      | Time to maturity.                                 |
| $r$      | Continuously compounded risk-free interest rate.  |
| $q$      | Continuous dividend yield.                        |
| $\sigma$ | Volatility, written mathematically as $\sigma$ .  |
| $N(x)$   | Standard normal cumulative distribution function. |

The continuous dividend yield  $q$  reduces the present value of the stock leg because dividends are paid out of the underlying asset over time. The risk-free rate  $r$  discounts the future strike payment or receipt back to present value.

## 2.3 Closed-Form European Prices

For  $T > 0$  and  $\sigma > 0$ , define

$$d_1 = \frac{\ln(S/K) + (r - q + \frac{1}{2}\sigma^2) T}{\sigma\sqrt{T}}, \quad (3)$$

$$d_2 = d_1 - \sigma\sqrt{T}. \quad (4)$$

The European call price with continuous dividends is

$$C = Se^{-qT}N(d_1) - Ke^{-rT}N(d_2). \quad (5)$$

The European put price with continuous dividends is

$$P = Ke^{-rT}N(-d_2) - Se^{-qT}N(-d_1). \quad (6)$$

Plain-language interpretation:

- $Se^{-qT}$  is the dividend-adjusted present value of the stock part.
- $Ke^{-rT}$  is the discounted strike part.
- $N(d_1)$  and  $N(d_2)$  are probability-weighted terms from the Black-Scholes model.
- The call formula compares the value of receiving the asset with paying the strike.
- The put formula compares the value of receiving the strike with giving up the asset.

## 2.4 Put-Call Parity

The call and put prices should satisfy put-call parity with continuous dividends:

$$C - P = Se^{-qT} - Ke^{-rT}. \quad (7)$$

This identity is a powerful consistency check. It says that a European call minus a European put with the same  $S$ ,  $K$ ,  $T$ ,  $r$ ,  $q$ , and  $\sigma$  should equal a simple discounted stock-minus-strike position. If the implemented call and put formulas violate this identity, at least one formula is wrong.

## 2.5 Zero-Volatility Deterministic Case

At  $\sigma = 0$ , the random diffusion vanishes. With positive maturity, the implementation uses the deterministic discounted payoff:

$$C_{\sigma=0} = \max \left( S e^{-qT} - K e^{-rT}, 0 \right), \quad (8)$$

$$P_{\sigma=0} = \max \left( K e^{-rT} - S e^{-qT}, 0 \right). \quad (9)$$

This special case is not optional. The usual  $d_1$  and  $d_2$  definitions divide by  $\sigma\sqrt{T}$ , so explicit handling avoids divide-by-zero behavior and makes the finance meaning clear.

## 2.6 Why This Is Not American Option Solving Yet

American options can be exercised before maturity. That early exercise right creates a payoff obstacle, a continuation region, an exercise region, and a free boundary. Those ideas are central to the later CN/P-SOR solver, but they are not implemented in Ticket 1. The functions in this ticket do not choose early exercise times and do not solve a variational inequality or linear complementarity problem.

# 3 Design Decisions

## 3.1 Package Namespace

The package namespace `src/american_risk_surfaces/` is used because Ticket 0 recommended a `src` layout with one explicit project namespace. This makes imports clear:

```
from american_risk_surfaces.solvers.black_scholes import european_call_price
```

This is safer than importing from a bare package name such as `solvers`, which could collide with another package or script name later.

## 3.2 Testing Framework

`unittest` is used instead of `pytest` because Ticket 0 found that `pytest` was not installed in the checked Python runtimes. `unittest` is part of the Python standard library, so it lets the project start testing without adding `pyproject.toml`, `requirements.txt`, or any dependency installation step.

## 3.3 Avoiding SciPy for Now

SciPy is avoided for now because Ticket 0 found that SciPy availability differs between runtimes. The only SciPy-like object needed in this ticket is the standard normal CDF  $N(x)$ . Python's standard `math.erf` function is enough for this small utility step because

$$N(x) = \frac{1}{2} \left[ 1 + \operatorname{erf} \left( \frac{x}{\sqrt{2}} \right) \right]. \quad (10)$$

This decision keeps the first code ticket portable. It is not a claim that the helper is a high-performance replacement for scientific statistics libraries.

### 3.4 Scalar and Array Inputs

Scalar and NumPy array inputs are both supported. Scalar inputs are convenient for simple examples, reports, and selected spot checks. Array inputs are important because later validation will compare many spot values across grids or vectors. Returning a Python `float` for scalar  $S$  and a NumPy array for array-like  $S$  makes the API natural in both situations.

### 3.5 Explicit Edge Cases

The edge cases  $T = 0$  and  $\sigma = 0$  are handled explicitly. At maturity, the correct answer is exactly payoff. With zero volatility and positive maturity, the stock path is deterministic under the model, so the option price becomes the deterministic discounted payoff shown above. These branches also prevent numerical warnings in exactly the cases where future validation needs clarity.

## 4 Implementation Record

### 4.1 Files Created

| File                                                                         | Role in Ticket 1                                                                     |
|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>src/american_risk_surfaces/__init__.py</code>                          | Marks <code>american_risk_surfaces</code> as the project package.                    |
| <code>src/american_risk_surfaces/solvers/__init__.py</code>                  | Marks <code>solvers</code> as a subpackage for analytic and future solver utilities. |
| <code>src/american_risk_surfaces/solvers/black_scholes.py</code>             | Contains the payoff functions and European Black-Scholes pricing utilities.          |
| <code>tests/test_black_scholes.py</code>                                     | Contains focused <code>unittest</code> coverage for the public utility behavior.     |
| <code>reports/03_solver/tickets/ticket_01_black_scholes_utilities.md</code>  | Keeps the Markdown study-report source from the earlier report revision.             |
| <code>reports/03_solver/tickets/ticket_01_black_scholes_utilities.tex</code> | LaTeX source for this formal academic study report.                                  |
| <code>reports/03_solver/tickets/ticket_01_black_scholes_utilities.pdf</code> | Formal PDF compiled from the LaTeX source.                                           |

### 4.2 Public Functions

| Function                 | Ticket 1 behavior and later use                                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>call_payoff</code> | Computes the immediate exercise payoff of a call, $\max(S - K, 0)$ , returning a float or NumPy array. Later this supports terminal call payoff values and future call obstacle checks.       |
| <code>put_payoff</code>  | Computes the immediate exercise payoff of a put, $\max(K - S, 0)$ , returning a float or NumPy array. Later this supports terminal put payoff values and future American put obstacle checks. |

| Function                         | Ticket 1 behavior and later use                                                                                                                                                                                                     |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>european_call_price</code> | Computes the European call price with continuous dividends, returning a closed-form value as a float or NumPy array. Later this is a benchmark for European call finite-difference validation and no-dividend American call checks. |
| <code>european_put_price</code>  | Computes the European put price with continuous dividends, returning a closed-form value as a float or NumPy array. Later this is the main benchmark for European put finite-difference validation.                                 |

All four functions validate obvious domain errors. Negative spot values are rejected, the strike must be positive, maturity must be nonnegative, and volatility must be nonnegative. For scalar spot input, the functions return a Python `float`. For array-like spot input, they return a NumPy array with the corresponding shape.

### 4.3 Function Notes

`call_payoff(S, K)`. The function computes the immediate exercise value of a call option. If  $S \leq K$ , the payoff is zero. If  $S > K$ , the payoff is  $S - K$ . This will later define call terminal values on a grid and support American call obstacle checks when those tickets arrive.

`put_payoff(S, K)`. The function computes the immediate exercise value of a put option. If  $S \geq K$ , the payoff is zero. If  $S < K$ , the payoff is  $K - S$ . This will later define put terminal values and support the future American put obstacle check  $U \geq \Phi_{\text{put}}$ .

`european_call_price(S, K, T, r, q, sigma)`. The function computes the European call price under the dividend-adjusted Black-Scholes formula. At  $T = 0$ , it returns  $\Phi_{\text{call}}$ . At  $\sigma = 0$  and  $T > 0$ , it returns the deterministic discounted call payoff. Later, this function will benchmark future European finite-difference call validation and no-dividend American call validation.

`european_put_price(S, K, T, r, q, sigma)`. The function computes the European put price under the dividend-adjusted Black-Scholes formula. At  $T = 0$ , it returns  $\Phi_{\text{put}}$ . At  $\sigma = 0$  and  $T > 0$ , it returns the deterministic discounted put payoff. Later, this function will be the main benchmark for European put finite-difference validation, which matters because American puts are the core early-exercise case.

## 5 Testing Strategy

The tests are deliberately focused on analytic utility behavior. They do not test a finite-difference solver. Table 4 summarizes the test categories and their meaning.

Table 4: Ticket 1 test categories and interpretation.

| Test category                     | What it proves                                                                                                | What it does not prove                                            |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Payoff tests                      | Call and put payoff signs are correct below, at, and above strike.                                            | Time value, discounting, or volatility behavior.                  |
| Scalar and array behavior         | Scalar inputs return floats and NumPy arrays preserve shape.                                                  | Performance for every possible array-like input.                  |
| $T = 0$ payoff equality           | The maturity boundary is handled explicitly.                                                                  | Positive-maturity pricing accuracy.                               |
| $\sigma = 0$ deterministic payoff | Zero-volatility logic avoids $d_1, d_2$ divide-by-zero behavior and uses discounted deterministic payoff.     | Any stochastic volatility behavior.                               |
| Put-call parity                   | Call and put formulas are internally consistent for continuous dividends.                                     | Every market convention or model extension.                       |
| Nonnegative price sanity checks   | Calls and puts are not negative in a representative vector case.                                              | Full arbitrage-free behavior in every parameter regime.           |
| Monotonicity checks               | Calls increase with $S$ and puts decrease with $S$ in a simple vector example.                                | A full mathematical proof of monotonicity.                        |
| Invalid input checks              | Negative spot, nonpositive strike, negative maturity, and negative volatility raise <code>ValueError</code> . | A full production validation policy for every unusual input type. |

The test suite uses real code and no mocks. This is appropriate because the four functions are small analytic utilities with deterministic outputs.

## 6 Test Results and Interpretation

The exact test command is:

```
PYTHONPYCACHEPREFIX=/private/tmp/codex_pycache PYTHONPATH=src python3 -m unittest
discover -s tests
```

The final observed result was:

```
Ran 9 tests in 0.001s

OK
```

In this context, “Ran 9 tests OK” means the Ticket 1 unit tests passed for the implemented payoff and European Black-Scholes utility functions. It means the tested payoff behavior, scalar and array behavior, maturity edge case, zero-volatility edge case, put-call parity identity, basic sanity checks, and invalid input checks all behaved as expected.

It does not mean the future finite-difference solver is validated. No spot grid, time grid, Black-Scholes finite-difference operator, boundary adjustment, Crank-Nicolson time step, PSOR projection, American obstacle enforcement, boundary extraction, or Greek calculation was tested in Ticket 1.



The correct interpretation is narrow but important: the project now has analytic utilities that are credible enough to use as benchmark functions in later validation tickets.

## 7 Limitations

This ticket only validates analytic utility behavior. It does not include:

- finite-difference grid construction;
- Black-Scholes finite-difference operator setup;
- Crank-Nicolson time stepping;
- PSOR projection;
- American option obstacle enforcement;
- continuation premium calculation;
- boundary extraction;
- Delta or Gamma calculations;
- numerical solver validation;
- experiments, datasets, stress maps, or neural-network files.

The normal-CDF implementation uses `math.erf` through NumPy vectorization. That is suitable for this small dependency-light utility layer, but it is not meant to be a high-performance replacement for scientific statistics libraries in a larger production system.

The tests use a small number of representative cases. They are designed to catch common formula and edge-case mistakes, not to certify every possible input regime.

## 8 Lessons Learned / Study Notes

The first lesson is that payoff is the foundation. Before discussing PDEs, free boundaries, or neural networks, students should be able to draw and explain  $\max(S - K, 0)$  for a call and  $\max(K - S, 0)$  for a put.

The second lesson is that European and American options are different validation tools. European options have closed-form Black-Scholes prices in this model, so they are useful for checking a numerical PDE solver. American options usually do not have the same simple closed-form solution because early exercise creates an obstacle problem.

The third lesson is that dividends matter. The dividend yield  $q$  changes the stock leg through  $Se^{-qT}$  and appears in  $d_1$  through  $r - q$ . A wrong sign on  $q$  is a common source of incorrect option prices and later solver errors.

The fourth lesson is that edge cases are not side issues. At  $T = 0$ , price equals payoff. At  $\sigma = 0$ , the uncertain stock movement disappears. If code ignores those cases, it may produce division-by-zero warnings or misleading values exactly where future validation needs clarity.

The fifth lesson is that a passing unit test suite should be interpreted carefully. Passing Ticket 1 tests is good evidence for these four analytic functions. It is not evidence that the future CN/PSOR solver, boundary extraction, or Greeks are correct.

For beginner researchers, the habit to build here is disciplined layering: validate the simple analytic layer before trusting more complicated numerical layers.

## 9 Link to Next Ticket

Ticket 1 prepares Ticket 2 by providing the payoff and closed-form benchmark utilities that later finite-difference work will need.

Ticket 2 should build the finite-difference grid and Black-Scholes operator setup. That means future code will begin representing spot points, time-to-maturity points, interior nodes, and the discrete version of the Black-Scholes spatial operator.

The connection is:

- Ticket 1 payoff functions can define terminal payoff values on a future grid.
- Ticket 1 European price functions can benchmark future European finite-difference values.
- Ticket 2 should not need to redefine payoff or closed-form Black-Scholes formulas.

Ticket 2 still should not implement Crank-Nicolson, PSOR, American option solving, boundary extraction, Greeks, experiments, datasets, stress maps, or neural networks unless a later reviewed ticket explicitly asks for those pieces.

## 10 Reproducibility Record

### 10.1 Files Changed

| Path                                                            | Status in this report task                                           |
|-----------------------------------------------------------------|----------------------------------------------------------------------|
| reports/03_solver/tickets/ticket_01_black_scholes_utilities.tex | Created as the formal LaTeX report source.                           |
| reports/03_solver/tickets/ticket_01_black_scholes_utilities.pdf | Overwritten by a PDF compiled from LaTeX.                            |
| reports/03_solver/tickets/ticket_01_black_scholes_utilities.md  | Kept as the Markdown study report source from the previous revision. |
| src/american_risk_surfaces/solvers/black_scholes.py             | Read for factual consistency; not changed.                           |
| tests/test_black_scholes.py                                     | Read for factual consistency; not changed.                           |

Source code change during this LaTeX formatting task: none.

Test files changed during this LaTeX formatting task: none.

### 10.2 Commands

Test command:

```
PYTHONPYCACHEPREFIX=/private/tmp/codex_pycache PYTHONPATH=src python3 -m unittest discover -s tests
```

Final test result:

```
Ran 9 tests in 0.001s
```

```
OK
```

Compile command for Python syntax:

```
PYTHONPYCACHEPREFIX=/private/tmp/codex_pycache PYTHONPATH=src python3 -m compileall  
-q src tests
```

LaTeX compile command:

```
HOME=/private/tmp FC_CACHEDIR=/private/tmp/fontconfig-cache latexmk -xelatex -  
interaction=nonstopmode -halt-on-error -outdir=/private/tmp/ticket01_latex  
reports/03_solver/tickets/ticket_01_black_scholes_utilities.tex
```

PDF verification used `pdftinfo` and rendered page images with `pdftoppm`. The PDF is a LaTeX-generated formal report on A4 paper with one-inch margins, page numbers, tables, monospace commands and paths, and LaTeX math.

### 10.3 Git Status Summary

At the final check for this LaTeX report task, the Ticket 1 files were still untracked:

```
?? reports/03_solver/tickets/ticket_01_black_scholes_utilities.md  
?? reports/03_solver/tickets/ticket_01_black_scholes_utilities.pdf  
?? reports/03_solver/tickets/ticket_01_black_scholes_utilities.tex  
?? src/  
?? tests/
```

## References

Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973. doi: 10.1086/260062.

Robert C. Merton. Theory of rational option pricing. *The Bell Journal of Economics and Management Science*, 4(1):141, 1973. doi: 10.2307/3003143.